

Softmax, Worked Out by Hand

How a language model turns a row of raw, unbounded scores (*logits*) into a proper probability distribution over the vocabulary — every exponential, sum and division shown on real numbers.

What this document does. It takes one tiny vocabulary of five tokens, fixes a concrete logit vector straight out of an LLM’s final layer, and computes the softmax *by hand* — exponentials, normaliser, probabilities — then proves the three properties that make softmax the right choice (sums to one, shift-invariance, monotonicity), shows the numerically stable form every real implementation uses, derives the Jacobian, and connects it to the cross-entropy training loss. Every number was produced by the NumPy snippet in Appendix A, so the arithmetic is exact and self-consistent.

Concepts covered, in order: logits vs. probabilities · the softmax formula · exponentiate–sum–normalise · the three defining properties · the max-subtraction trick for numerical stability · the temperature knob · the softmax Jacobian · softmax + cross-entropy.

1 The problem: scores are not probabilities

The last linear layer of a language model emits one real number per vocabulary token. These are *logits*: unbounded, signed, and on no particular scale. For a toy vocabulary {cat, sat, on, the, mat} the model might output

$$\mathbf{z} = (2.0, 1.0, 0.1, -1.0, 3.0).$$

We cannot read these as “probabilities”: they are negative in places and they do not sum to one. We need a map $\mathbb{R}^K \rightarrow \mathbb{R}^K$ that (i) makes every entry positive, (ii) makes them sum to one, and (iii) preserves the *order* so the biggest logit stays the most likely token. Softmax is exactly that map.

Intuition

Think of each logit as a *vote in log-space*. Exponentiating turns a difference of logits into a *ratio* of probabilities: a token whose logit is 1 larger than another’s becomes $e^1 \approx 2.72\times$ more likely, no matter where on the number line the two logits sit. So softmax cares only about the *gaps* between scores, never their absolute level — which is exactly why temperature (scaling the gaps) is such a clean control knob later on.

2 The formula

For a logit vector $\mathbf{z} = (z_1, \dots, z_K) \in \mathbb{R}^K$, the softmax produces a vector $\mathbf{p} = (p_1, \dots, p_K)$ with

$$p_i = \text{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \quad i = 1, \dots, K.$$

The numerator e^{z_i} makes every entry strictly positive. The denominator $Z = \sum_j e^{z_j}$ — the *partition function* or normaliser — is the same for every i , and dividing by it forces $\sum_i p_i = 1$. Three operations, in order: **exponentiate, sum, normalise**.

Mini-example · two logits collapse to the sigmoid

With $\mathbf{z} = (2, 1)$:

$$e^2 = 7.389, \quad e^1 = 2.718, \quad Z = 10.107,$$

$$p_1 = \frac{7.389}{10.107} = 0.731, \quad p_2 = \frac{2.718}{10.107} = 0.269.$$

Notice $p_1 = \frac{e^2}{e^2 + e^1} = \frac{1}{1 + e^{-(2-1)}} = \sigma(1) = 0.731$. **Softmax over two classes is the logistic sigmoid of the logit difference** — the binary case is not a special rule, it falls out of the same formula.

3 Worked by hand on the five-token vector

Step 1 — exponentiate each logit.

token	cat	sat	on	the	mat
z_i	2.0	1.0	0.1	-1.0	3.0
e^{z_i}	7.3891	2.7183	1.1052	0.3679	20.0855

Step 2 — sum to get the normaliser.

$$Z = 7.3891 + 2.7183 + 1.1052 + 0.3679 + 20.0855 = 31.6659.$$

Step 3 — divide each exponential by Z .

token	cat	sat	on	the	mat
p_i	0.2333	0.0858	0.0349	0.0116	0.6343

Check: $0.2333 + 0.0858 + 0.0349 + 0.0116 + 0.6343 = 1.0000$. ✓ The model assigns **63.4%** to **mat** — the largest logit (3.0) became the dominant probability, while the negative logit (-1.0, **the**) is crushed to 1.2%.

Intuition

The gap from **mat** (3.0) to **cat** (2.0) is only one logit unit, yet **mat** ends up $\approx 2.7\times$ more probable (0.634 vs 0.233). One unit of logit = one factor of e in probability *ratio*. That exponential stretch is what makes a language model commit to a token rather than hedge evenly.

4 The three properties that justify softmax

(1) It is a valid distribution. Each $p_i = e^{z_i}/Z > 0$ since exponentials are positive and $Z > 0$, and

$$\sum_{i=1}^K p_i = \frac{\sum_i e^{z_i}}{\sum_j e^{z_j}} = \frac{Z}{Z} = 1.$$

So $\mathbf{p}$ lives on the probability simplex — no entry can leak below 0 or above 1.

(2) Shift-invariance. Add the *same* constant c to every logit and nothing changes:

$$\text{softmax}(\mathbf{z} + c\mathbf{1})_i = \frac{e^{z_i+c}}{\sum_j e^{z_j+c}} = \frac{e^c e^{z_i}}{e^c \sum_j e^{z_j}} = \frac{e^{z_i}}{\sum_j e^{z_j}} = \text{softmax}(\mathbf{z})_i.$$

The e^c cancels top and bottom. Softmax reads *differences*, not absolute levels — the algebraic statement of the intuition above, and the licence for the stability trick in the next section.

(3) Monotonicity / order preservation. If $z_a > z_b$ then $e^{z_a} > e^{z_b}$, hence $p_a > p_b$. The ranking of logits is the ranking of probabilities, so $\arg \max$ is preserved: softmax never reorders the model's preferences, it only sharpens them.

Mini-example • shift-invariance on the real vector

Subtract the max ($c = -3.0$) from every logit, then redo softmax:

$$\mathbf{z} - 3.0 = (-1.0, -2.0, -2.9, -4.0, 0.0), \quad e^{(\cdot)} = (0.3679, 0.1353, 0.0550, 0.0183, 1.0),$$

$$Z' = 1.5766, \quad \mathbf{p} = (0.2333, 0.0858, 0.0349, 0.0116, 0.6343).$$

Identical to Step 3 — but now the largest exponential is exactly $e^0 = 1$ instead of 20.09. That is the whole idea behind numerical stability.

5 Numerical stability: subtract the max

A raw logit of, say, 1000 overflows `float` when exponentiated ($e^{1000} = \infty$). Shift-invariance lets us fix this for free: subtract $m = \max_i z_i$ from every logit *before* exponentiating. The result is provably identical, but now the largest exponent is $e^0 = 1$ and all others lie in $(0, 1]$, so nothing overflows:

$$p_i = \frac{e^{z_i - m}}{\sum_j e^{z_j - m}}, \quad m = \max_k z_k$$

This is the form in every production library (PyTorch, NumPy, JAX). The mathematics is unchanged; only the floating-point path is made safe.

6 The temperature knob

Scale the logits by $1/T$ before softmax:

$$p_i(T) = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}.$$

T stretches or compresses the *gaps* between logits, and because softmax reads gaps, T directly controls how sharp or flat the distribution is. On our vector:

T	cat	sat	on	the	mat	behaviour
0.5	0.1170	0.0158	0.0026	0.0003	0.8643	sharper, more confident
1.0	0.2333	0.0858	0.0349	0.0116	0.6343	the raw distribution
2.0	0.2587	0.1569	0.1001	0.0577	0.4266	flatter, more diverse
100	0.2019	0.1999	0.1981	0.1960	0.2040	nearly uniform

As $T \rightarrow 0$ softmax approaches a one-hot $\arg \max$ (greedy decoding); as $T \rightarrow \infty$ it approaches the uniform distribution $1/K = 0.2$. (*Temperature gets its own full treatment in PDF #2; here it is just softmax with rescaled inputs.*)

7 The Jacobian (for backprop)

Training needs $\partial p_i / \partial z_j$. Differentiating $p_i = e^{z_i} / Z$:

$$\frac{\partial p_i}{\partial z_j} = p_i(\delta_{ij} - p_j), \quad \delta_{ij} = \begin{cases} 1 & i = j \\ 0 & i \neq j \end{cases}$$

so the diagonal is $p_i(1 - p_i)$ and the off-diagonal is $-p_i p_j$. Compactly, $J = \text{diag}(\mathbf{p}) - \mathbf{p}\mathbf{p}^\top$.

Mini-example · Jacobian on $\mathbf{z} = (1, 2, 0.5)$

Here $\mathbf{p} = (0.2312, 0.6285, 0.1402)$, so

$$J = \begin{pmatrix} 0.1778 & -0.1453 & -0.0324 \\ -0.1453 & 0.2335 & -0.0881 \\ -0.0324 & -0.0881 & 0.1206 \end{pmatrix}.$$

Each row sums to 0 (e.g. $0.1778 - 0.1453 - 0.0324 \approx 0$): pushing one probability up must pull the others down, since they are tied together by the normaliser. The matrix is symmetric and negative off-diagonal — competition between tokens, made explicit.

8 Why it pairs with cross-entropy

In training, the target is the true next token. With one-hot target on class t , the cross-entropy loss is $\mathcal{L} = -\log p_t$. Composing it with the softmax gives the famously clean gradient

$$\frac{\partial \mathcal{L}}{\partial z_j} = p_j - y_j,$$

i.e. *predicted minus target* — the messy exponentials cancel. On our example, if the correct token is **mat** ($p = 0.6343$) the loss is $-\log 0.6343 = 0.4552$; if the correct token were instead **on** ($p = 0.0349$) the loss would be $-\log 0.0349 = 3.3552$. The model is penalised in proportion to *how much probability mass it withheld from the truth* — and softmax is what made that mass a well-defined probability in the first place.

Equation cheat-sheet

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}} \quad (\text{stable}) \quad p_i = \frac{e^{z_i - m}}{\sum_j e^{z_j - m}}, \quad m = \max_k z_k$$

$$\sum_i p_i = 1, \quad \text{softmax}(\mathbf{z} + c\mathbf{1}) = \text{softmax}(\mathbf{z}), \quad z_a > z_b \Rightarrow p_a > p_b$$

$$p_i(T) = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}, \quad \frac{\partial p_i}{\partial z_j} = p_i(\delta_{ij} - p_j), \quad \frac{\partial(-\log p_t)}{\partial z_j} = p_j - y_j$$

Appendix A · Reference implementation

```
import numpy as np

def softmax(z, T=1.0):
    z = np.asarray(z, float) / T
    z = z - z.max() # shift-invariance => numerical stability
    e = np.exp(z)
    return e / e.sum()

z = np.array([2.0, 1.0, 0.1, -1.0, 3.0])
p = softmax(z) # [0.2333 0.0858 0.0349 0.0116 0.6343]
assert abs(p.sum() - 1.0) < 1e-12
print(softmax(z, T=0.5)) # sharper: mat -> 0.8643
print(softmax(z, T=2.0)) # flatter: mat -> 0.4266
```

Internal learning material. Numbers reproducible from the appendix code. v1.0